Programación de Dispositivos Móviles

Smart Home

Memoria Técnica del Proyecto

Valentín Pedrosa Campoy

CONTENIDOS

1. Introducción **3**
1.1 Descripción del proyecto 3
1.2 Objetivos técnicos 3
1.3 Stack tecnológico 3
1.4 Estructura del repositorio 4
2. Arquitectura de software **4**
2.1 Patrón arquitectónico: Hexagonal con cortes verticales 4
Arquitectura Hexagonal (Ports & Adapters) 4
Vertical slicing (división por funcionalidad) 5
Resultado práctico de la combinación 5
2.2 Principios SOLID 6
S - Single Responsibility Principle 6
O - Open/Closed Principle 7
L - Liskov Substitution Principle 7
I - Interface Segregation Principle 7
D - Dependency Inversion Principle 8
2.3 Capas dentro de cada slice 8
2.4 Modelo de dominio 9
Device: interfaz sellada 9
Identidades con value classes 10
DeviceEvent 10
2.5 Puertos (interfaces de dominio) 10
Puertos de repositorio 10
Puerto de control de dispositivos 11
Puertos de comisionamiento y descubrimiento 11
2.6 Casos de uso (capa de aplicación) 11
ToggleDeviceUseCase 11
BulkToggleDevicesByTypeUseCase 12
CommissionDeviceUseCase 13
2.7 Adaptadores (capa de infraestructura) 13
InMemoryDeviceRepository 13
MatterDeviceControlAdapter 14
Adaptadores fake para tests 14
2.8 ViewModels: puente entre dominio y UI 14
DashboardViewModel 14
VoiceControlViewModel 15
2.9 Inyección de dependencias con Koin 15
Organización en módulos 15
single vs factory: ciclo de vida de las dependencias 16

Registro diferenciado por plataforma: platformModule 16
Inyección en ViewModels 17
El simulador de sensores: singleton con scope propio 17
2.10 Estructura de paquetes completa 18
2.11 Tests 18
3. Protocolo Matter: integración y decisiones técnicas 19
3.1 Qué es Matter 19
3.2 Arquitectura de comunicación 19
3.3 PASE vs CASE 20
Por qué no se usa CASE 20
3.4 Flujo de comisionamiento 20
3.5 Flujo de control de dispositivos 21
3.6 Expiración de sesiones PASE y solución con nodeId temporal 21
3.7 Timeout de comandos 22
3.8 Operaciones que usan Matter vs. simulación local 22
3.9 Dos modos de comisionamiento 23
3.10 Archivos clave de la integración Matter 23
4. Descubrimiento de dispositivos: mDNS 24
4.1 Contexto y decisión de diseño 24
4.2 Adaptador localhost con probe TCP (entorno emulador) 24
4.3 Adaptador orquestador con fallback automático 25
4.4 Adaptador mDNS para dispositivos reales 25
Extracción del discriminador 25
Gestión del ciclo de vida 25
4.5 Adaptador mDNS para localizar el simulador 26
4.6 Por qué mDNS falla en el emulador 28
4.7 Resumen de decisiones 28
5. Simulación de dispositivos Matter 28
5.1 Tecnología utilizada: matter.js 28
5.2 Dispositivos simulados 29
Bombillas inteligentes (10) - DimmableLight 29
Interruptores on/off (5) - OnOffPlugInUnit 29
Cerraduras (2) - DoorLock 29
Sensor de contacto (1) - ContactSensor 30
Persianas (4) - WindowCovering 30
Smart TV (1) - BasicVideoPlayer 30
Sensores de medición (3) y termostato (1) 30
5.3 Estructura del código de simulación 30
5.4 Arranque del simulador 31
6. Simulación de sensores y sistema de eventos 32
6.1 Por qué los sensores se simulan localmente 32

6.2 SensorEventSimulator 32
Lecturas de temperatura 32
Eventos de sensor de contacto (puerta) 32
Alertas de humo 33
Alertas de fugas de agua 33
Ajustes de termostato 33
Monitor de duración de puerta abierta 33
Simulación de presencia (antiokupas) 33
6.3 Generación de IDs de eventos 33
6.4 Flujo de eventos hacia la UI 33
6.5 Notificaciones Android 33
7. App companion Wear OS 34
7.1 Propósito y funcionalidad 34
Acciones disponibles por tipo de dispositivo 34
7.2 Decisión de diseño: acciones rápidas vs. control por voz 34
7.3 Arquitectura de la app Wear OS 34
7.4 Protocolo de comunicación Watch–Phone 35
Mensajes del protocolo 35
Flujo completo de una interacción 35
7.5 Recepción en el teléfono 35
7.6 Fallback con datos simulados 36
7.7 Limitaciones conocidas 36
8. Control por voz 37
8.1 Diseño del sistema de reconocimiento 37
8.2 Reconocimiento de voz: AndroidSpeechRecognizerAdapter 37
8.3 Parsing de comandos: ParseVoiceCommandUseCase 37
Comandos soportados 38
Extracción de habitación 38
Resultado del parsing 38
8.4 Ejecución: ExecuteVoiceCommandUseCase 39
8.5 Ejemplo de flujo completo 39
9. Funcionalidades principales 39
9.1 Control de dispositivos 39
Escalas de traducción Matter 40
9.2 Gestión de habitaciones 40
9.3 Modo antiokupas 40
9.4 Notificaciones y alertas 40
9.5 Internacionalización 41
9.6 Dashboard principal 41
9.7 Tests unitarios 41
10. Resumen de decisiones técnicas 41

10.1 Kotlin Multiplatform sobre desarrollo nativo separado 41
10.2 Arquitectura hexagonal con vertical slicing 42
10.3 PASE en lugar de CASE para la comunicación Matter 42
10.4 NodeId temporal para forzar nuevas sesiones PASE 42
10.5 Simulación local de eventos de sensores 42
10.6 Tres adaptadores de descubrimiento intercambiables 42
10.7 Acciones rápidas en Wear OS en lugar de control por voz 43
10.8 Parsing de voz mediante regex sin ML 43
10.9 Timeout explícito de 3 segundos para comandos Matter 43
10.10 No actualizar estado en memoria si falla Matter 43
11. Integración continua (CI/CD) 44
11.1 Pipeline de GitHub Actions 44
Eventos de activación 44
11.2 Jobs 44
Job test - Tests unitarios 44
Job build - Compilación de APKs 44
11.3 Decisiones de diseño del pipeline 44

1. Introducción

1.1 Descripción del proyecto

Smart Home es una aplicación multiplataforma de gestión de casa inteligente desarrollada con **Kotlin Multiplatform** y **Compose Multiplatform**. Permite controlar dispositivos del hogar en tiempo real desde un teléfono Android y un smartwatch Wear OS. La arquitectura Kotlin Multiplatform hace que la lógica de negocio y la UI sean también compilables para iOS, aunque la integración con el protocolo Matter es exclusiva de Android (el SDK de Matter solo está disponible en androidMain).

Smart Home

El proyecto simula un entorno domótico completo con **27 dispositivos Matter virtuales** distribuidos por habitaciones: bombillas inteligentes, persianas motorizadas, cerraduras electrónicas, interruptores, una Smart TV, un termostato y varios sensores (temperatura, humo, fugas de agua, contacto de puerta).

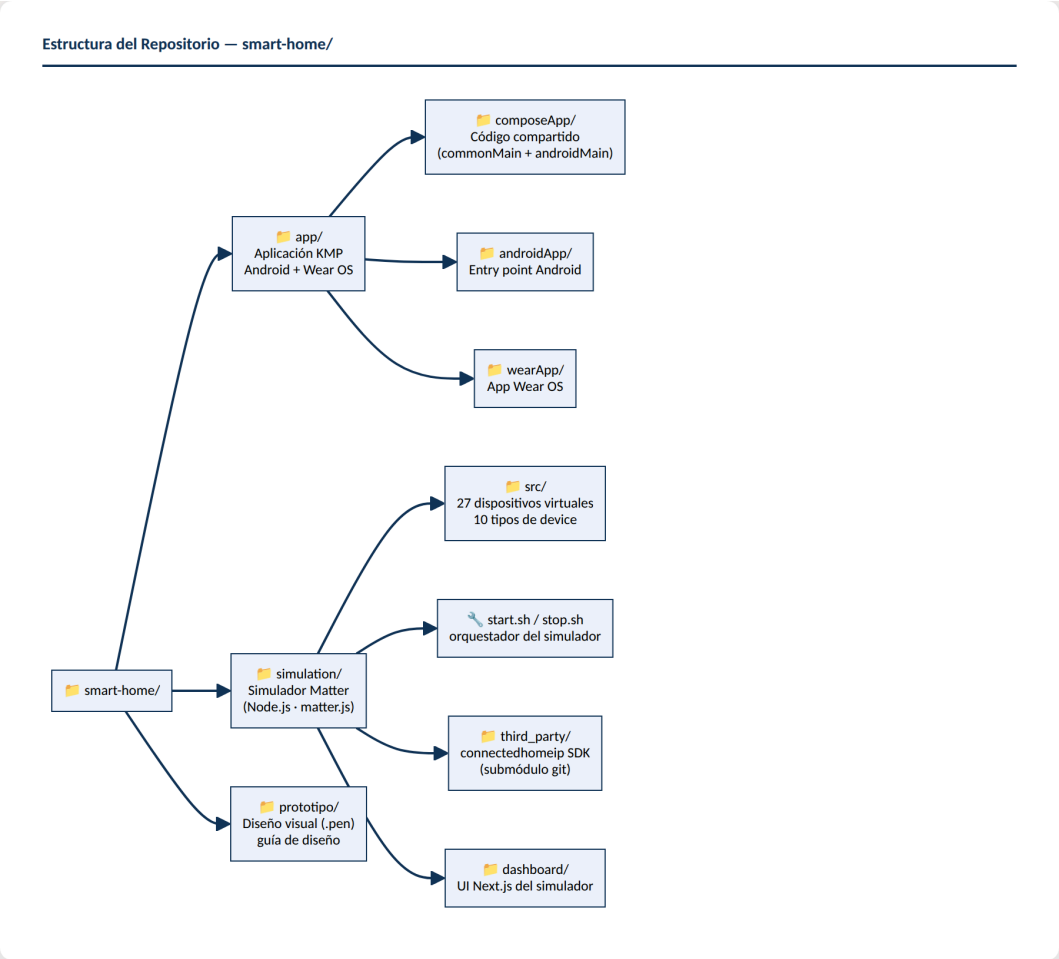
1.2 Objetivos técnicos

- Demostrar la viabilidad de Kotlin Multiplatform para aplicaciones de domótica multiplataforma.
- Integrar el protocolo **Matter** (anteriormente Project CHIP) como capa de comunicación estándar entre la app y dispositivos simulados.
- Explorar las limitaciones del SDK de demostración de Matter en entornos de emulación Android.
- Implementar una arquitectura limpia, testeable y extensible mediante el patrón hexagonal con cortes verticales.
- Proporcionar múltiples interfaces de control: pantalla táctil, voz y smartwatch.

1.3 Stack tecnológico

Componente	Tecnología
Lenguaje principal	Kotlin 2.3
UI multiplataforma	Compose Multiplatform 1.10
Sistema de diseño	Material 3
Inyección de dependencias	Koin 4.1
Programación reactiva	Kotlin Flows
Serialización	kotlinx.serialization
Navegación	Jetpack Navigation
Wear OS	Wear Compose Material 3
Comunicación smartwatch	Google Wearable Data Layer API
Protocolo IoT	Matter / connectedhomeip
Simulador de dispositivos	matter.js (Node.js v24+)
Reconocimiento de voz	Android SpeechRecognizer (nativo)

1.4 Estructura del repositorio



Estructura del repositorio smart-home

2. Arquitectura de software

2.1 Patrón arquitectónico: Hexagonal con cortes verticales

La aplicación combina dos estrategias complementarias:

Arquitectura Hexagonal (Ports & Adapters)

Propuesta por Alistair Cockburn (2005), la arquitectura hexagonal organiza el sistema en dos zonas estrictamente separadas:

- **Núcleo** (domain/ + application/): contiene las reglas de negocio y los casos de uso. No importa ninguna clase de Android, Matter, Koin ni de cualquier otro framework. Es Kotlin puro, ejecutable en cualquier JVM.
- **Exterior** (infrastructure/ + ui/): contiene los adaptadores que conectan el núcleo con el mundo real: base de datos, protocolo Matter, sistema operativo Android, pantalla.

La comunicación entre ambas zonas se realiza exclusivamente a través de **puertos**: interfaces definidas en el dominio que el exterior implementa. El núcleo nunca depende del exterior; es siempre al revés.

Vertical slicing (división por funcionalidad)

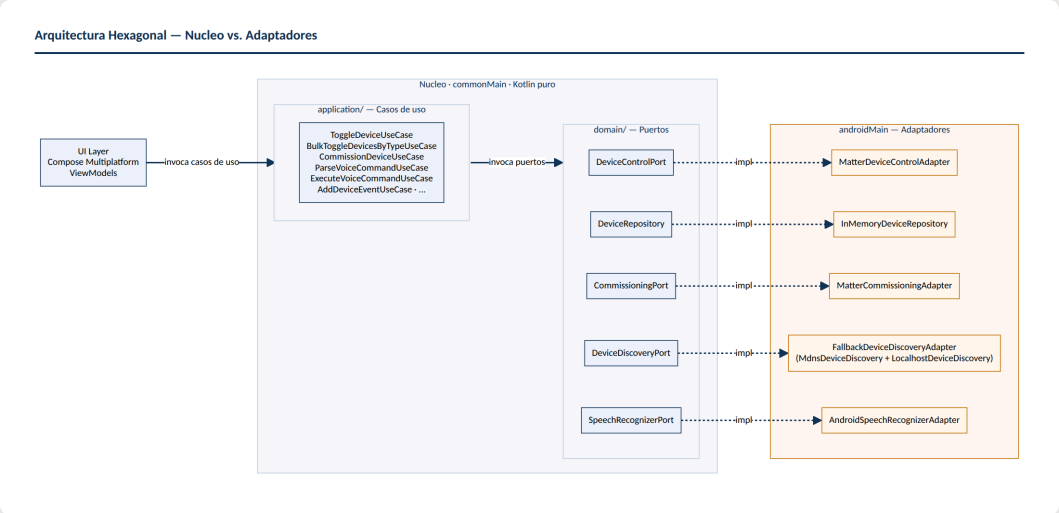
En lugar de organizar el código en capas horizontales globales (models/, services/, repositories/), cada funcionalidad es un **slice** autónomo con sus propias capas internas. Hay siete slices:

Slice	Responsabilidad
device	Núcleo canónico: entidades (Device, DeviceId, RoomId, Color…), puertos y repositorios usados por el resto de slices
commissioning	Descubrimiento mDNS y emparejamiento PASE con dispositivos Matter
voice	Parsing y ejecución de comandos de voz
event	Eventos de sensores, notificaciones, historial
settings	Configuración de la app
antisquatter	Simulación de presencia para disuadir intrusiones
room	Gestión de habitaciones

Cada slice puede evolucionar, añadir dependencias o reemplazar adaptadores sin afectar a los demás.

Resultado práctico de la combinación

- La lógica de negocio en domain/ y application/ no importa ninguna clase de Android, Matter ni de ningún framework.
- Los adaptadores de infraestructura (Matter, notificaciones, reconocimiento de voz) están aislados en infrastructure/ bajo el source set androidMain/.
- Más de 93 tests unitarios se ejecutan en la JVM en segundos, sin emulador ni dispositivo físico, porque el núcleo no tiene dependencias de plataforma.



Arquitectura hexagonal: núcleo vs. adaptadores

2.2 Principios SOLID

La arquitectura hexagonal facilita de forma natural el respeto de los principios SOLID. A continuación se muestra cómo se manifiesta cada uno en el código real del proyecto.

S - Single Responsibility Principle

Cada clase tiene una única razón para cambiar. Los casos de uso son el ejemplo más directo: una clase, una responsabilidad.

```
// Una sola responsabilidad: hacer toggle de UN dispositivo
class ToggleDeviceUseCase(...)

// Una sola responsabilidad: hacer toggle masivo por tipo
class BulkToggleDevicesByTypeUseCase(...)

// Una sola responsabilidad: hacer toggle masivo por tipo EN una habitación
class BulkToggleDevicesByTypeInRoomUseCase(...)
```

Lo mismo aplica a los repositorios (persistir/observar dispositivos), adaptadores (traducir comandos de dominio a llamadas Matter), y ViewModels (transformar Flows de dominio en estado de UI).

InMemoryDeviceRepository solo sabe gestionar el almacén de dispositivos en memoria. No sabe nada de la UI, ni de Matter, ni de la lógica de negocio:

```
class InMemoryDeviceRepository : DeviceRepository {
    private val store = MutableStateFlow(emptyMap<DeviceId, Device>())

    override fun observeAllDevices(): Flow<List<Device>> =
        store.map { it.values.toList() }

    override suspend fun save(device: Device) {
        store.update { it + (device.id to device) }
```

```
    }
    // ...
}
```

O - Open/Closed Principle

Las clases están **abiertas a extensión** y **cerradas a modificación**. Añadir un nuevo tipo de dispositivo no requiere modificar los casos de uso existentes gracias a la interfaz sellada Device y el despacho polimórfico:

```
// ToggleDeviceUseCase no necesita cambiar si se añade, por ejemplo, un
// nuevo tipo AirPurifier, solo hay que añadirlo al when exhaustivo.
when (device) {
    is Light     -> deviceControlPort.toggleOnOff(id, !device.isOn)
    is Lock      -> deviceControlPort.lockDoor(id, !device.isLocked)
    is Switch    -> deviceControlPort.toggleOnOff(id, !device.isOn)
    is SmartTv   -> deviceControlPort.toggleOnOff(id, !device.isOn)
    is Thermostat -> deviceControlPort.setThermostatMode(id, !device.isHeatingOn)
    else -> return
}
```

Del mismo modo, añadir un nuevo adaptador de descubrimiento (por ejemplo, basado en Bluetooth en lugar de mDNS) no requiere tocar ningún caso de uso: basta con crear una nueva clase que implemente DeviceDiscoveryPort.

L - Liskov Substitution Principle

Cualquier implementación de un puerto puede sustituirse por otra sin que el código que la usa cambie. Esto se verifica de forma práctica en los tests: FakeDeviceControlPort sustituye a MatterDeviceControlAdapter sin que ToggleDeviceUseCase note la diferencia.

```
// En producción (androidMain), Koin inyecta el adaptador real:
single<DeviceControlPort> { MatterDeviceControlAdapter(get()) }

// En tests, se instancia directamente el fake:
val controlPort = FakeDeviceControlPort()
val useCase = ToggleDeviceUseCase(repo, controlPort, eventRepo)
// El caso de uso funciona igual con ambos
```

Del mismo modo, LocalhostDeviceDiscoveryAdapter, MdnsDeviceDiscoveryAdapter y FallbackDeviceDiscoveryAdapter son intercambiables como implementaciones de DeviceDiscoveryPort; el CommissioningViewModel no sabe ni necesita saber cuál está activo.

I - Interface Segregation Principle

Los puertos son interfaces pequeñas y cohesivas, no monolitos. Cada puerto agrupa solo las operaciones que pertenecen conceptualmente al mismo contrato:

```
// Puerto de descubrimiento: solo descubrir
interface DeviceDiscoveryPort {
    fun discoverDevices(): Flow<List<DiscoveredDevice>>
```

```
}

// Puerto de comisionamiento: solo comisionar
interface CommissioningPort {
    suspend fun commission(device: DiscoveredDevice): Result<Device>
}

// Puerto de localización del simulador: solo localizar su IP
interface SimulatorDiscoveryPort {
    suspend fun discoverSimulatorHost(): String?
}
```

Si se necesitase un cliente que solo descubre dispositivos, no necesita depender de CommissioningPort. Si se necesitase solo comisionar, no necesita depender de DeviceDiscoveryPort. Cada consumidor depende solo de lo que usa.

D - Dependency Inversion Principle

Este principio es el corazón de la arquitectura hexagonal: los módulos de alto nivel (casos de uso) no dependen de los módulos de bajo nivel (Matter SDK, Android APIs), ambos dependen de abstracciones (puertos).

```
// ToggleDeviceUseCase depende de la abstracción DeviceControlPort,
// nunca de MatterDeviceControlAdapter directamente.
class ToggleDeviceUseCase(
    private val deviceRepository: DeviceRepository,    // abstracción
    private val deviceControlPort: DeviceControlPort,  // abstracción
    private val deviceEventRepository: DeviceEventRepository, // abstracción
)

// CommissionDeviceUseCase tampoco sabe que existe Matter:
class CommissionDeviceUseCase(
    private val commissioningPort: CommissioningPort,  // abstracción
    private val deviceRepository: DeviceRepository,   // abstracción
    private val deviceControlPort: DeviceControlPort, // abstracción
)
```

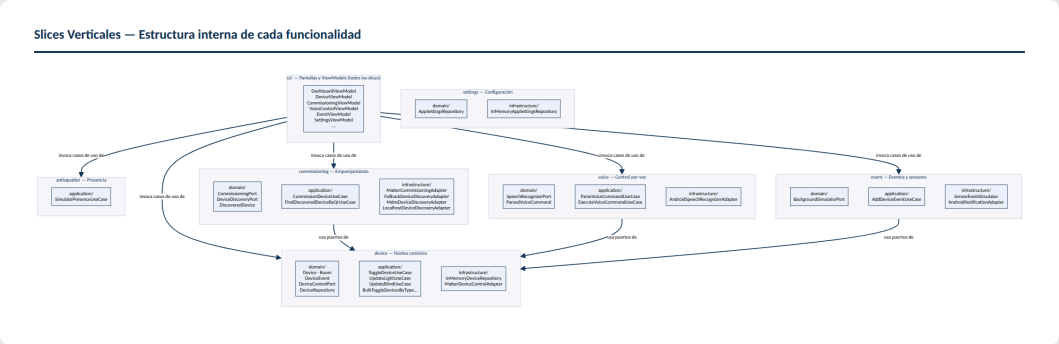
La inversión la materializa Koin en tiempo de ejecución, inyectando las implementaciones concretas (ver sección 2.9).

2.3 Capas dentro de cada slice

Cada slice vertical sigue la misma estructura interna:

```
<feature>/
├── domain/
│   ├── model/          # Entidades y value objects del slice
│   ├── <Port>.kt       # Interfaces de puerto (en la raíz de domain/)
│   └── <Repository>.kt # Interfaces de repositorio
├── application/
│   └── <Verb><Noun>UseCase.kt  # Un caso de uso por clase
└── infrastructure/
    ├── persistence/    # Implementaciones de repositorio
    └── <adapter>/      # Otros adaptadores de infraestructura
```

Regla de dependencias: las capas internas nunca dependen de las externas. domain/ no conoce application/; application/ no conoce infrastructure/. La UI (ui/<feature>/) tampoco vive dentro de los slices, sino en un módulo independiente que sí puede importar casos de uso.



Slices verticales y sus capas internas

2.4 Modelo de dominio

El modelo de dominio está en device/domain/model/. El slice device actúa como **núcleo canónico**: sus entidades (Device, DeviceId, RoomId, Color, DeviceEvent...) son importadas por todos los demás slices. Esta colocación refleja que device es la razón de ser de la app, no un slice más.

Device: interfaz sellada

Device es una **interfaz sellada** que agrupa todos los tipos de dispositivo. Cada subtipo es un data class inmutable:

```
sealed interface Device {
    val id: DeviceId
    val name: String
    val roomId: RoomId?
    val type: DeviceType
}

// Ejemplo de subtipo - todos siguen el mismo patrón
data class Light(
    override val id: DeviceId,
    override val name: String,
    override val roomId: RoomId?,
    val isOn: Boolean,
    val color: Color,
    val brightness: Int,
) : Device {
    override val type get() = DeviceType.LIGHT
    fun toggle(): Light = copy(isOn = !isOn)
}

// Resto de subtipos: Lock, Blind, Switch, SmartTv, Thermostat,
// TemperatureSensor, SmokeAlarm, WaterLeakSensor, ContactSensor
```

Los dispositivos son **inmutables**: toda modificación devuelve una copia nueva (copy()). Esto hace el estado predecible y los tests sencillos.

Identidades con value classes

Los IDs son **inline value classes**, lo que proporciona type-safety sin coste en tiempo de ejecución:

```
@JvmInline value class DeviceId(val value: String)
@JvmInline value class RoomId(val value: String)
```

El compilador impide pasar un RoomId donde se espera un DeviceId, aunque ambos sean String por dentro.

DeviceEvent

```
data class DeviceEvent(
    val id: String,
    val deviceId: DeviceId,
    val type: DeviceEventType,
    val message: String,
    val timestamp: Instant,
)

enum class DeviceEventType {
    SMOKE_ALERT, WATER_LEAK_ALERT, TEMPERATURE_READING,
    DOOR_OPENED, DOOR_CLOSED, DOOR_OPEN_TOO_LONG,
    THERMOSTAT_ADJUSTED, DEVICE_TURNED_ON, DEVICE_TURNED_OFF,
}
```

2.5 Puertos (interfaces de dominio)

Los puertos definen lo que el núcleo **necesita** del exterior, sin saber cómo se implementa. Hay tres tipos:

Puertos de repositorio

Contratos para acceder al estado persistido, expuestos como Flows reactivos:

```
interface DeviceRepository {
    fun observeAllDevices(): Flow<List<Device>>
    fun observeDevice(id: DeviceId): Flow<Device?>
    fun observeDevicesByRoom(roomId: RoomId): Flow<List<Device>>
    fun observeDevicesByType(type: DeviceType): Flow<List<Device>>
    suspend fun save(device: Device)
    suspend fun saveAll(devices: List<Device>)
    suspend fun delete(id: DeviceId)
}

interface RoomRepository {
```

```
    fun observeAllRooms(): Flow<List<Room>>
    fun observeRoom(id: RoomId): Flow<Room?>
    suspend fun save(room: Room)
    suspend fun delete(id: RoomId)
}

interface DeviceEventRepository {
    fun observeAllEvents(): Flow<List<DeviceEvent>>
    fun observeEventsByDevice(deviceId: DeviceId): Flow<List<DeviceEvent>>
    suspend fun add(event: DeviceEvent)
    suspend fun clear()
}
```

Puerto de control de dispositivos

El contrato que el dominio impone a cualquier tecnología de control (Matter, fake, etc.):

```
interface DeviceControlPort {
    fun registerDevice(deviceId: DeviceId, connectionInfo: DeviceConnectionInfo)
    fun deregisterDevice(deviceId: DeviceId)
    suspend fun toggleOnOff(deviceId: DeviceId, on: Boolean)
    suspend fun setLevel(deviceId: DeviceId, level: Int)
    suspend fun lockDoor(deviceId: DeviceId, lock: Boolean)
    suspend fun setThermostatSetpoint(deviceId: DeviceId, temperatureCelsius: Double)
    suspend fun setThermostatMode(deviceId: DeviceId, heating: Boolean)
    suspend fun setWindowCoveringPosition(deviceId: DeviceId, openPercent: Int)
    suspend fun launchContent(deviceId: DeviceId, url: String)
}
```

Puertos de comisionamiento y descubrimiento

```
interface DeviceDiscoveryPort {
    fun discoverDevices(): Flow<List<DiscoveredDevice>>
}

interface CommissioningPort {
    suspend fun commission(device: DiscoveredDevice): Result<Device>
}
```

2.6 Casos de uso (capa de aplicación)

Los casos de uso orquestan el dominio: leen del repositorio, invocan puertos y persisten los cambios. Siguen estas convenciones:

- Un archivo, una clase: <Verbo><Sustantivo>UseCase.
- Punto de entrada único: operator fun invoke(...) para llamarlos como funciones.
- No importan nada de Android ni de frameworks de UI.

ToggleDeviceUseCase

Ilustra el patrón completo: Matter primero, estado en memoria solo si Matter tiene éxito:

```
class ToggleDeviceUseCase(
    private val deviceRepository: DeviceRepository,
    private val deviceControlPort: DeviceControlPort,
    private val deviceEventRepository: DeviceEventRepository,
) {
    suspend operator fun invoke(id: DeviceId) {
        val device = deviceRepository.observeDevice(id).first() ?: return

        // 1. Enviar comando Matter - es la fuente de verdad
        try {
            when (device) {
                is Light  -> deviceControlPort.toggleOnOff(id, !device.isOn)
                is Lock   -> deviceControlPort.lockDoor(id, !device.isLocked)
                is Switch -> deviceControlPort.toggleOnOff(id, !device.isOn)
                is SmartTv -> deviceControlPort.toggleOnOff(id, !device.isOn)
                is Thermostat -> deviceControlPort.setThermostatMode(id, !device.isHeatingOn)
                else -> return   // sensores son solo lectura
            }
        } catch (_: Exception) {
            // Matter falló → NO actualizar estado en memoria
            return
        }

        // 2. Matter tuvo éxito → actualizar estado local
        val toggled = when (device) {
            is Light  -> device.toggle()
            is Lock   -> device.toggle()
            is Switch -> device.toggle()
            is SmartTv -> device.toggle()
            is Thermostat -> device.toggleHeating()
            else -> return
        }
        deviceRepository.save(toggled)

        // 3. Si es cerradura, registrar evento en historial
        if (device is Lock) {
            deviceEventRepository.add(DeviceEvent(
                id = Uuid.random().toString(),
                deviceId = id,
                type = if (!device.isLocked) DeviceEventType.DOOR_CLOSED
                       else DeviceEventType.DOOR_OPENED,
                message = "...",
                timestamp = Clock.System.now(),
            ))
        }
    }
}
```

BulkToggleDevicesByTypeUseCase

Demuestra la reutilización de lógica de dominio compartida (toggleDevice):

```
class BulkToggleDevicesByTypeUseCase(
    private val deviceRepository: DeviceRepository,
    private val deviceControlPort: DeviceControlPort,
) {
    suspend operator fun invoke(type: DeviceType, turnOn: Boolean): Int {
        val devices = deviceRepository.observeDevicesByType(type).first()
```

```
        val updated = devices.mapNotNull { device ->
            toggleDevice(device, turnOn, deviceControlPort)  // función de dominio compartida
        }
        if (updated.isNotEmpty()) deviceRepository.saveAll(updated)
        return updated.size
    }
}
```

CommissionDeviceUseCase

Coordina tres puertos: comisionamiento, registro de conexión y persistencia:

```
class CommissionDeviceUseCase(
    private val commissioningPort: CommissioningPort,
    private val deviceRepository: DeviceRepository,
    private val deviceControlPort: DeviceControlPort,
) {
    suspend operator fun invoke(device: DiscoveredDevice, customName: String?): Result<Device> {
        return commissioningPort.commission(device).map { commissioned ->
            val named = if (!customName.isNullOrBlank()) commissioned.withName(customName)
                        else commissioned
            deviceControlPort.registerDevice(named.id, DeviceConnectionInfo(
                host = device.host, port = device.port, passcode = device.passcode,
            ))
            deviceRepository.save(named)
            named
        }
    }
}
```

2.7 Adaptadores (capa de infraestructura)

Los adaptadores implementan los puertos del dominio con tecnologías concretas.

InMemoryDeviceRepository

Implementa DeviceRepository usando un MutableStateFlow<Map<DeviceId, Device>>. Cualquier llamada a save() actualiza el mapa atómicamente y todos los colectores del Flow reciben el nuevo estado de forma automática, sin polling:

```
class InMemoryDeviceRepository : DeviceRepository {
    private val store = MutableStateFlow(emptyMap<DeviceId, Device>())

    override fun observeAllDevices(): Flow<List<Device>> =
        store.map { it.values.toList() }

    override suspend fun save(device: Device) =
        store.update { it + (device.id to device) }

    // observeDevice, observeDevicesByRoom, saveAll, delete...
}
```

MatterDeviceControlAdapter

Implementa DeviceControlPort con el SDK de Matter, incluyendo el retry con nodeId temporal (ver sección 3.6):

```
// androidMain
class MatterDeviceControlAdapter(
    private val chipController: ChipDeviceController,
) : DeviceControlPort {
    // Caché de device pointers: nodeId → pointer
    private val pointerCache = mutableMapOf<Long, Long>()
    private val mutex = Mutex()

    override suspend fun toggleOnOff(deviceId: DeviceId, on: Boolean) =
        executeWithRetry(deviceId) { pointer ->
            val cluster = ChipClusters.OnOffCluster(chipController, pointer, ENDPOINT)
            if (on) cluster.on(...) else cluster.off(...)
        }

    // ... resto de operaciones
}
```

Adaptadores fake para tests

En commonTest/ se usan implementaciones fake inyectadas por Koin:

```
class FakeDeviceControlPort : DeviceControlPort {
    val toggledDevices = mutableListOf<Pair<DeviceId, Boolean>>()

    override suspend fun toggleOnOff(deviceId: DeviceId, on: Boolean) {
        toggledDevices += deviceId to on
    }
    // ... resto de métodos sin efecto
}
```

2.8 ViewModels: puente entre dominio y UI

Los ViewModels reciben los puertos/casos de uso por inyección de dependencias y exponen un único StateFlow<UiState> a los Composables. Usan combine para fusionar múltiples Flows de dominio en un único estado de UI.

DashboardViewModel

```
class DashboardViewModel(
    deviceRepository: DeviceRepository,
    roomRepository: RoomRepository,
    deviceEventRepository: DeviceEventRepository,
) : ViewModel() {

    val uiState: StateFlow<DashboardUiState> = combine(
        deviceRepository.observeAllDevices(),
        roomRepository.observeAllRooms(),
        deviceEventRepository.observeAllEvents(),
```

```
    ) { devices, rooms, events ->
        DashboardUiState(/* calcula estado a partir de los tres flows */)
    }.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), DashboardUiState())
}
```

El stateIn con WhileSubscribed(5_000) cancela la suscripción al Flow si no hay colectores durante 5 segundos (por ejemplo, la app está en segundo plano), ahorrando recursos.

VoiceControlViewModel

Conecta tres Flows del SpeechRecognizerPort con los casos de uso de voz:

```
class VoiceControlViewModel(
    private val speechRecognizer: SpeechRecognizerPort,
    private val parseVoiceCommand: ParseVoiceCommandUseCase,
    private val executeVoiceCommand: ExecuteVoiceCommandUseCase,
    private val voiceCommandRepository: VoiceCommandRepository,
) : ViewModel() {

    init {
        speechRecognizer.recognizedText
            .onEach { text -> processCommand(text) }
            .launchIn(viewModelScope)
    }

    private fun processCommand(text: String) {
        viewModelScope.launch {
            val parsed = parseVoiceCommand(text)           // dominio puro
            val result = executeVoiceCommand(parsed, text) // ejecuta sobre dispositivos
            voiceCommandRepository.add(VoiceCommand(text, result, Clock.System.now()))
        }
    }
}
```

2.9 Inyección de dependencias con Koin

La inyección de dependencias se gestiona con **Koin**, un framework ligero para Kotlin que no usa reflexión en tiempo de compilación. Su papel es materializar el principio de inversión de dependencias: las interfaces están definidas en el dominio, y Koin conecta en tiempo de ejecución cada interfaz con su implementación concreta.

Organización en módulos

El grafo de dependencias se divide en tres módulos Koin:

- **deviceModule** (commonMain): repositorios, casos de uso y simuladores. Todo el código multiplataforma.
- **platformModule** (androidMain): adaptadores Android-específicos (Matter, notificaciones, SpeechRecognizer, mDNS).
- **uiModule** (commonMain): ViewModels.

Esta separación es importante: deviceModule puede compilarse y testearse sin ninguna dependencia de Android.

single vs factory: ciclo de vida de las dependencias

Koin distingue dos ciclos de vida fundamentales:

```
val deviceModule = module {

    // single → una sola instancia compartida (repositorios, estado)
    single<DeviceRepository>        { InMemoryDeviceRepository() }
    single<RoomRepository>          { InMemoryRoomRepository() }
    single<DeviceEventRepository>   { InMemoryDeviceEventRepository() }
    single<AntiSquatterRepository>  { InMemoryAntiSquatterRepository() }
    single<AppSettingsRepository>   { InMemoryAppSettingsRepository() }
    single<VoiceCommandRepository>  { InMemoryVoiceCommandRepository() }
    single<BackgroundSimulatorPort> { SensorEventSimulator(get(), get(), get(), scope = ...) }

    // factory → nueva instancia cada vez que se solicita (casos de uso, stateless)
    factory { ToggleDeviceUseCase(get(), get(), get()) }
    factory { CommissionDeviceUseCase(get(), get(), get()) }
    factory { ParseVoiceCommandUseCase() }
    factory { ExecuteVoiceCommandUseCase(get(), get(), get(), get(), get(), get(), get()) }
    // ... más de 20 casos de uso adicionales
}
```

Los repositorios son single porque son el estado de la aplicación: si hubiera dos instancias de InMemoryDeviceRepository, los cambios de un ViewModel no se propagarían a otro. Los casos de uso son factory porque son stateless (sin estado propio) y no necesitan ser compartidos.

Registro diferenciado por plataforma: platformModule

Los adaptadores que usan APIs de Android se registran en un módulo separado en androidMain/, que Koin carga junto con deviceModule:

```
// androidMain/di/PlatformModule.android.kt
actual val platformModule = module {

    single { MatterControllerProvider(androidContext()) }
     single<DeviceControlPort>  { MatterDeviceControlAdapter(get<MatterControllerProvider>().controller)
}
     single<CommissioningPort>  { MatterCommissioningAdapter(get<MatterControllerProvider>().controller)
}
    single<NotificationPort>   { AndroidNotificationAdapter(androidContext()) }
    single<SimulatorHostRepository>  { InMemorySimulatorHostRepository() }
    single<SimulatorDiscoveryPort>   { MdnsSimulatorDiscoveryAdapter(androidContext()) }

    // DeviceDiscoveryPort: mDNS en dispositivo físico, fallback TCP en emulador
    single<DeviceDiscoveryPort> {
        FallbackDeviceDiscoveryAdapter(
            primary          = MdnsDeviceDiscoveryAdapter(androidContext()),
            fallback         = LocalhostDeviceDiscoveryAdapter(),
            fallbackDelayMs  = 10_000L,
        )
    }

    // SpeechRecognizerPort: real si disponible, fake en caso contrario
    single<SpeechRecognizerPort> {
        if (SpeechRecognizer.isRecognitionAvailable(androidContext()))
            AndroidSpeechRecognizerAdapter(androidContext())
```

```
        else
            FakeSpeechRecognizer(CoroutineScope(SupervisorJob() + Dispatchers.Default))
    }
}
```

Este módulo no existe en commonMain: el compilador garantiza que ningún código multiplataforma importa las clases de Android.

Inyección en ViewModels

Los ViewModels se registran con el DSL viewModelOf, que resuelve el constructor automáticamente por tipo sin necesidad de declarar cada parámetro:

```
// uiModule (commonMain)
val uiModule = module {
    viewModelOf(::DashboardViewModel)
    viewModelOf(::VoiceControlViewModel)
    viewModelOf(::CommissioningViewModel)
    viewModelOf(::SettingsViewModel)
    // ... más ViewModels
}
```

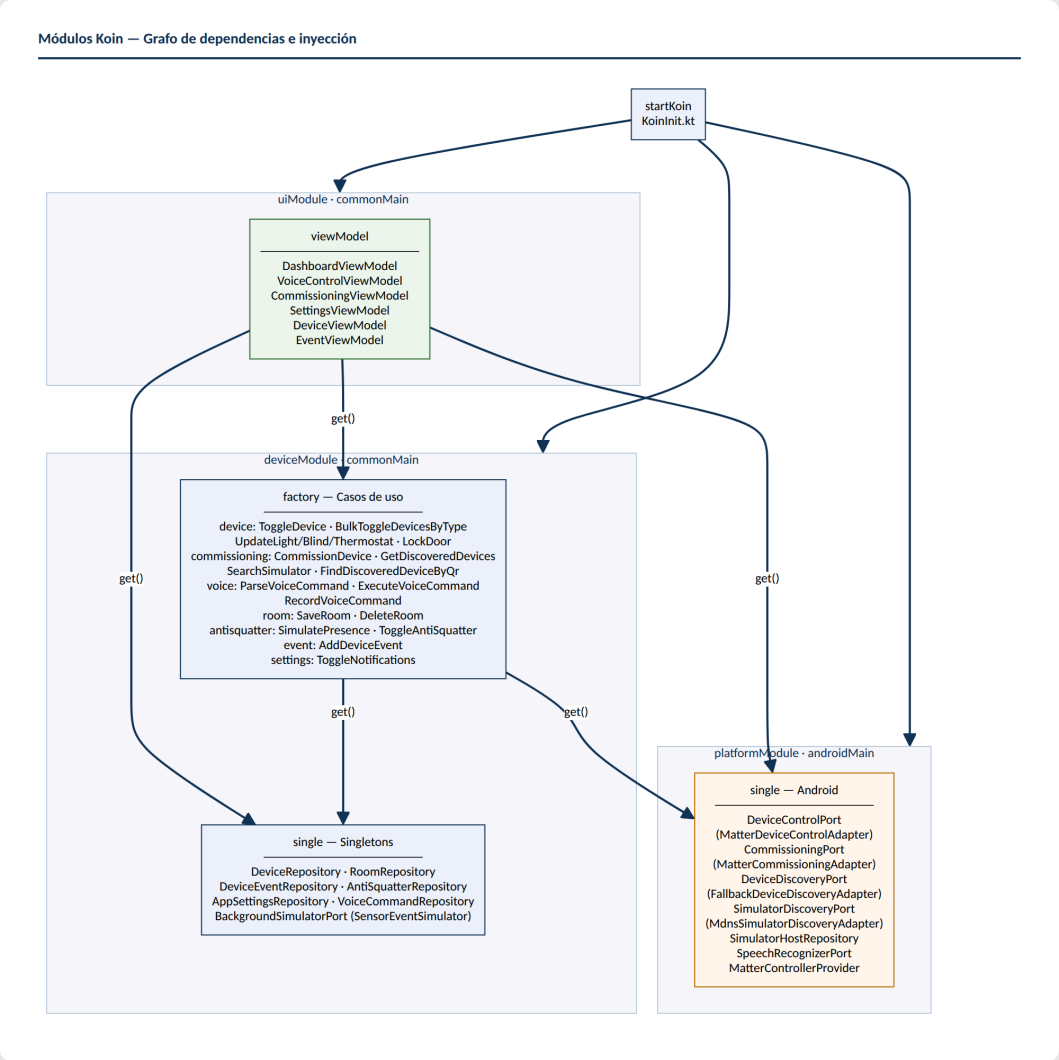
El SettingsViewModel es especialmente representativo: recibe SimulatorDiscoveryPort (que por debajo es MdnsSimulatorDiscoveryAdapter), pero no lo sabe. Cuando el usuario pulsa "buscar simulador", el ViewModel llama a simulatorDiscoveryPort.discoverSimulatorHost() y persiste el resultado. Todo el código de mDNS queda encapsulado en el adaptador.

El simulador de sensores: singleton con scope propio

El SensorEventSimulator es un caso especial: es un singleton que necesita su propio CoroutineScope independiente del ciclo de vida de cualquier ViewModel:

```
single<BackgroundSimulatorPort> {
    SensorEventSimulator(
        addDeviceEvent   = get(),         // AddDeviceEventUseCase
        deviceRepository = get(),         // DeviceRepository
        simulatePresence = get(),         // SimulatePresenceUseCase
        scope = CoroutineScope(SupervisorJob() + Dispatchers.Default),
    )
}
```

El SupervisorJob garantiza que si una de las siete corutinas del simulador falla, las demás continúan. El scope vive mientras viva el contenedor Koin (toda la vida de la app), lo que es correcto: los sensores deben emitir aunque el usuario cambie de pantalla.



Módulos Koin y grafo de dependencias

2.10 Estructura de paquetes completa

La estructura de paquetes sigue el patrón com.vpedrosa.smarthome.<slice>/<layer>/. Cada slice tiene sus propias capas domain/, application/ e infrastructure/, y la capa de UI vive en ui/<feature>/ separada de la lógica de dominio.

2.11 Tests

Los tests están en commonTest/ y se organizan siguiendo la misma estructura de slices (ver sección 2.1). Cada test instancia directamente los casos de uso con repositorios en memoria.

Cada test instancia directamente los casos de uso con repositorios en memoria y un FakeDeviceControlPort, sin mocks ni frameworks de mocking. El resultado es que los tests son legibles, deterministas y extremadamente rápidos (más de 93 tests en menos de 5 segundos en la JVM).

Ejemplo de test típico:

```
class ToggleDeviceUseCaseTest {
    private val repo = InMemoryDeviceRepository(listOf(
        Light(DeviceId("l1"), "Luz", null, isOn = false, ...)
    ))
    private val controlPort = FakeDeviceControlPort()
    private val eventRepo = InMemoryDeviceEventRepository()
    private val useCase = ToggleDeviceUseCase(repo, controlPort, eventRepo)

    @Test
    fun `toggle light sends Matter command and updates state`() = runTest {
        useCase(DeviceId("l1"))
        val device = repo.observeDevice(DeviceId("l1")).first()
        assertTrue((device as Light).isOn)
        assertTrue(controlPort.toggledDevices.any { it.first.value == "l1" && it.second })
    }

    @Test
    fun `failed Matter command does not update state`() = runTest {
        controlPort.shouldThrow = true
        useCase(DeviceId("l1"))
        val device = repo.observeDevice(DeviceId("l1")).first()
        assertFalse((device as Light).isOn)  // estado sin cambiar
    }
}
```

3. Protocolo Matter: integración y decisiones técnicas

3.1 Qué es Matter

Matter (anteriormente Project CHIP) es un estándar abierto de domótica promovido por la Connectivity Standards Alliance. Define un protocolo de comunicación unificado sobre IP que permite que dispositivos de distintos fabricantes interoperen sin dependencia de nubes propietarias.

La pila Matter incluye:

- **Descubrimiento**: mDNS/DNS-SD para anunciar y localizar dispositivos en la red local.
- **Comisionamiento**: proceso de emparejamiento seguro mediante PASE (con código de acceso) o CASE (con certificados).
- **Control**: comandos sobre clusters estandarizados (OnOff, DoorLock, WindowCovering, Thermostat...) enviados sobre UDP/IP.

3.2 Arquitectura de comunicación

La app utiliza el SDK de demostración de Matter para Android (matter-android-demo-sdk:1.0), que envuelve la librería nativa CHIPController. La comunicación con el simulador se realiza exclusivamente mediante **sesiones PASE**.

3.3 PASE vs CASE

	PASE	CASE
Propósito	Comisionamiento (emparejamiento)	Control continuo post-comisionamiento
Duración	Temporal (minutos)	Larga duración
Autenticación	Passcode (código numérico)	Certificados (fabric/NOC)
Estado en el proyecto	Único método utilizado	No soportado

Por qué no se usa CASE

La decisión de prescindir de CASE no es arbitraria; responde a tres limitaciones concretas encontradas durante el desarrollo:

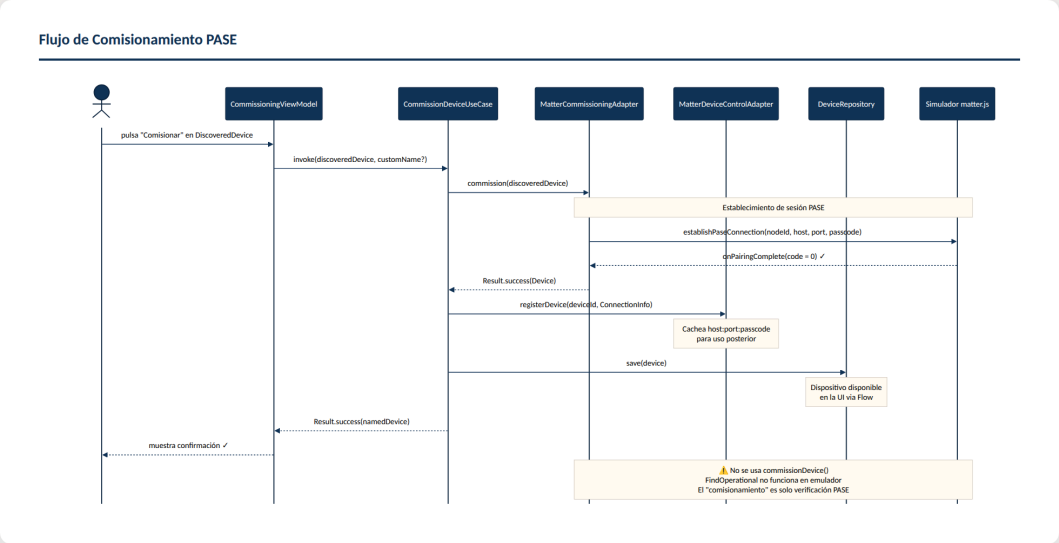
- **FindOperational (mDNS/CASE) no funciona en el emulador Android**: el descubrimiento operacional post-comisionamiento, que usa mDNS para localizar el dispositivo ya comisionado en la fabric, falla de forma sistemática en el entorno del emulador. El emulador Android tiene una pila de red virtualizada que no propaga correctamente las consultas mDNS necesarias para CASE.
- **No se realiza comisionamiento completo**: la app usa establishPaseConnection() en lugar de commissionDevice(). El flujo completo de comisionamiento incluye la generación de certificados NOC (Node Operational Certificate) y la incorporación del dispositivo a una fabric de confianza. Sin ese paso, no existe el material criptográfico necesario para establecer sesiones CASE.
- **Limitaciones del SDK de demostración**: el matter-android-demo-sdk:1.0 está diseñado para demostraciones y no expone todas las capacidades del SDK de producción de Google Home.

En consecuencia, el "comisionamiento" que realiza la app es en realidad una **verificación de conectividad PASE**: confirma que el dispositivo es alcanzable y acepta el passcode, pero no lo incorpora a ninguna fabric.

3.4 Flujo de comisionamiento

Los valores por defecto asignados en el comisionamiento son:

- Bombillas: brillo al 100%, color neutro.
- Sensores de temperatura: 21 °C.
- Termostato: 20 °C, modo calefacción desactivado.
- Persianas: posición 0% (cerradas).
- Cerraduras: estado bloqueado.



Flujo de comisionamiento PASE

3.5 Flujo de control de dispositivos

Una vez comisionado un dispositivo, cada interacción del usuario sigue este flujo:

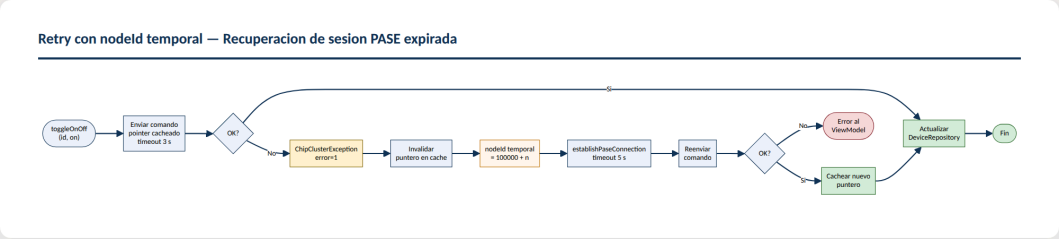
La decisión de **no actualizar el estado en caso de fallo** es deliberada: mantener la consistencia entre el estado en memoria de la app y el estado real del simulador es prioritario. Un optimistic update fallido que no se revierte generaría una UI inconsistente.

3.6 Expiración de sesiones PASE y solución con nodeId temporal

Las sesiones PASE son temporales por diseño en el estándar Matter. Cuando el simulador las expira por inactividad, los comandos subsiguientes fallan con ChipClusterException: CHIP cluster error: 1, y el simulador registra Ignoring message for unknown session.

El problema es que llamar a chipController.establishPaseConnection(nodeId) con el **mismo nodeId** no crea una sesión nueva: el controller reutiliza internamente la sesión existente (mismo LSID), aunque el simulador ya la haya descartado.

Solución implementada: el retry usa un **nodeId temporal incremental** (a partir de 100000) que fuerza al controller a crear una sesión PASE completamente nueva. El simulador acepta la conexión basándose únicamente en el passcode, independientemente del nodeId. El device pointer resultante se cachea contra el nodeId original del dispositivo para usos futuros.



Mecanismo de retry con nodeld temporal

3.7 Timeout de comandos

Los comandos Matter tienen un timeout de **3 segundos**. Sin este timeout explícito, el SDK reintenta internamente hasta 4 veces con backoff exponencial, lo que puede provocar esperas de ~15 segundos antes de detectar que la sesión expiró. El timeout de 3 segundos garantiza que el retry con sesión nueva se inicia en tiempo razonable.

El timeout de establecimiento PASE es de **5 segundos**.

3.8 Operaciones que usan Matter vs. simulación local

Operación	Usa Matter	Descripción
Comisionamiento	Sí (PASE)	Verifica conectividad y obtiene el device pointer
Toggle luz / switch / TV	Sí (PASE)	Envía comando OnOff cluster al simulador
Lock / Unlock cerradura	Sí (PASE)	Envía comando DoorLock cluster al simulador
Ajustar termostato	Sí (PASE)	Envía setpoint y modo de calefacción
Mover persiana	Sí (PASE)	Envía posición WindowCovering (escala 0-10000)
Brillo de bombilla	Sí (PASE)	Envía nivel LevelControl (escala 0-254)
Eventos de sensores	No	Generados localmente por SensorEventSimulator
Notificaciones push	No	Generadas localmente por AndroidNotificationAdapter
Lectura de temperatura	No	Valor aleatorio, no lectura real del sensor
Estado de puerta (contacto)	No	Toggle automático local, no lectura real

Operaciones Matter (PASE) vs. Simulación local

Operación	Canal	Descripción técnica
Comisionamiento	Matter	Verificación de conectividad PASE. No hay commissionDevice() completo; solo establece sesión con passcode.
Toggle luz / interruptor / TV	Matter	Comando OnOff cluster al simulador matter.js. Actualiza estado en DeviceRepository solo si Matter responde OK.
Brillo de bombilla	Matter	Comando LevelControl cluster. Escala UI 0–100 % → escala Matter 0–254.
Lock / Unlock cerradura	Matter	Comando DoorLock cluster. Genera evento DOOR_OPENED / DOOR_CLOSED en DeviceEventRepository.
Posición de persiana	Matter	Comando WindowCovering cluster. Escala UI 0–100 % → escala Matter 0–10 000.
Ajuste de termostato	Matter	Setpoint en 1/100 °C (ej. 22 °C → 2200). Modo calefacción vía ThermostatMode.
Eventos de sensores	Local	Generados por SensorEventSimulator con corutinas. Razón: las suscripciones de atributos requieren CASE, no soportado.
Lectura de temperatura	Local	Valores aleatorios 18–28 °C cada 3 s. No es lectura real del cluster TemperatureMeasurement del simulador.
Estado de puerta (contacto)	Local	Toggle automático cada 10 s por SensorEventSimulator. Modifica DeviceRepository directamente.
Notificaciones push	Local	Generadas por AndroidNotificationAdapter al recibir eventos del DeviceEventRepository. Sin servidor externo.

Operaciones Matter vs. simulación local

3.9 Dos modos de comisionamiento

La app distingue dos contextos de ejecución con adaptadores diferentes:

Adaptador	Contexto	Flujo
MatterCommissioningAdapter	Emulador / simulador	Solo establishPaseConnection() - sin NOC, sin fabric
ProductionCommissioningAdapter	Dispositivo físico real	establishPaseConnection() + commissionDevice() - NOC completo, fabric, CASE

ProductionCommissioningAdapter usa un timeout de 60 segundos para el proceso completo de comisionamiento. MatterCommissioningAdapter es el que se activa en el módulo de inyección de dependencias para el entorno habitual de desarrollo con el simulador.

3.10 Archivos clave de la integración Matter

Archivo	Responsabilidad
MatterControllerProvider.kt	Crea el ChipDeviceController singleton con configuración Android
MatterCommissioningAdapter.kt	Establece sesión PASE (comisionamiento ligero para emulador/simulador)
ProductionCommissioningAdapter.kt	Comisionamiento Matter completo con NOC y CASE para dispositivo físico
MatterDeviceControlAdapter.kt	Envía comandos a dispositivos con retry automático
SimulatorDeviceList.kt	Espejo de config.mjs: genera los 27 DiscoveredDevice a partir de host + índice

LocalhostDeviceDiscoveryAdapter.kt	Prueba TCP al simulador y emite la lista estática si responde
FallbackDeviceDiscoveryAdapter.kt	Decorador: lanza mDNS primero; si no hay dispositivos tras 10 s, activa localhost
MdnsDeviceDiscoveryAdapter.kt	Descubrimiento real de dispositivos Matter via NsdManager (_matter._tcp)
MdnsSimulatorDiscoveryAdapter.kt	Localiza el simulador via mDNS (_smarthome-hub._tcp) con fallback TCP
SensorEventSimulator.kt	Genera eventos de sensores localmente sin Matter

4. Descubrimiento de dispositivos: mDNS

4.1 Contexto y decisión de diseño

El descubrimiento de dispositivos es el primer paso para poder comisionar un dispositivo Matter. El estándar Matter utiliza **mDNS/DNS-SD** para que los dispositivos no comisionados se anuncien en la red local con el tipo de servicio _matterc._udp (o _matter._tcp en versiones anteriores), incluyendo en los registros TXT atributos como el discriminador.

La app implementa **cuatro adaptadores de descubrimiento** en el slice commissioning:

Adaptador	Rol	Mecanismo
LocalhostDeviceDiscoveryAdapter	Fallback para emulador / adb forward	Probe TCP al puerto 8085 del simulador; emite lista estática si responde
MdnsDeviceDiscoveryAdapter	Descubrimiento en dispositivo físico	NsdManager de Android, tipo _matter._tcp
FallbackDeviceDiscoveryAdapter	Orquestador (compuesto)	Lanza mDNS; si no hay dispositivos en 10 s, activa LocalhostDeviceDiscoveryAdapter
MdnsSimulatorDiscoveryAdapter	Localización del hub simulador	NsdManager con tipo _smarthome-hub._tcp + fallback TCP

En producción, el módulo DI registra FallbackDeviceDiscoveryAdapter como implementación de DeviceDiscoveryPort. El resto de adaptadores son sus estrategias internas o se usan de forma independiente para la localización del simulador.

La separación en adaptadores distintos respeta el principio de inversión de dependencias: los casos de uso del slice commissioning solo conocen DeviceDiscoveryPort y son agnósticos al mecanismo concreto de descubrimiento.

4.2 Adaptador localhost con probe TCP (entorno emulador)

LocalhostDeviceDiscoveryAdapter resuelve el problema de que el emulador Android no propaga mDNS. En lugar de mantener una lista codificada que se emite incondicionalmente, el adaptador **comprueba primero si el simulador está realmente en marcha** haciendo una conexión TCP al puerto 8085 (el WebSocket del simulador):

Los candidatos se prueban en orden:

Prioridad	Host	Contexto
1	10.0.2.2	localhost de la máquina host desde el emulador Android
2	127.0.0.1	loopback, útil con adb forward en dispositivo físico

Si la conexión TCP tiene éxito, llama a SimulatorDeviceList.forHost(host) para generar los 27 DiscoveredDevice con el host correcto. Si ningún candidato responde, el flow termina sin emitir.

SimulatorDeviceList es un objeto separado que refleja simulation/src/config.mjs: calcula puerto, discriminador, passcode y número de serie por índice para los mismos 27 dispositivos.

4.3 Adaptador orquestador con fallback automático

FallbackDeviceDiscoveryAdapter implementa el patrón **Decorator/Strategy** para combinar ambos adaptadores sin que los casos de uso lo sepan:

El flow resultante permanece abierto mientras la pantalla de comisionamiento esté visible; el job mDNS se cancela automáticamente al desaparecer el consumidor. Esta estrategia garantiza que en un dispositivo físico en la misma red que el simulador se use mDNS, y en un emulador (donde mDNS no funciona) se use la sonda TCP sin cambiar una sola línea de código del ViewModel ni del caso de uso.

4.4 Adaptador mDNS para dispositivos reales

MdnsDeviceDiscoveryAdapter usa el API NsdManager de Android para descubrir dispositivos Matter en la red local. Es el adaptador utilizado con dispositivos físicos.

Extracción del discriminador

El discriminador se obtiene del campo D en los registros TXT del anuncio mDNS. El valor es un entero de 12 bits que identifica unívocamente al dispositivo durante el proceso de comisionamiento y permite filtrar dispositivos en entornos con múltiples dispositivos Matter activos.

Gestión del ciclo de vida

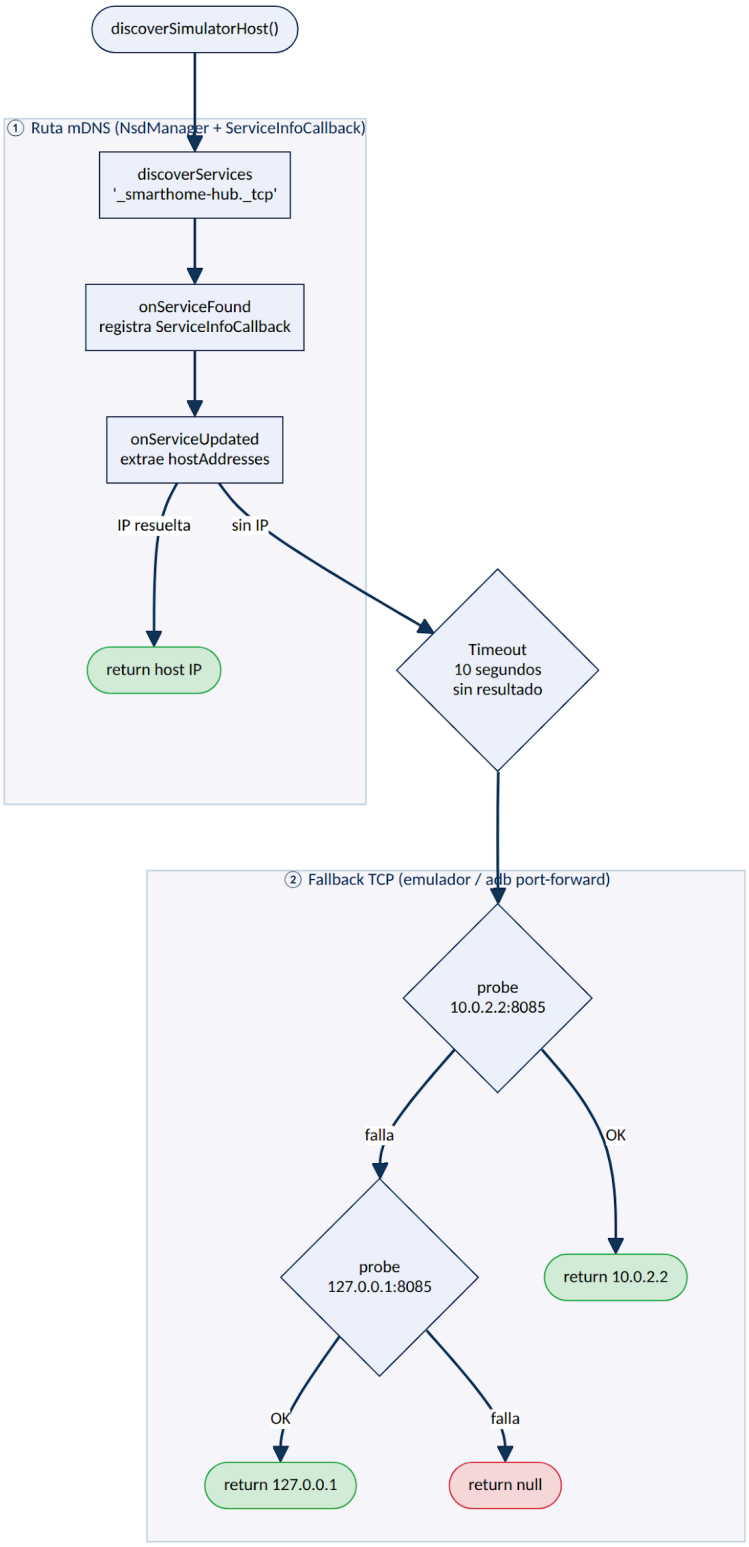
El adaptador expone un Flow<List<DiscoveredDevice>> que emite cada vez que cambia la lista de dispositivos descubiertos. Los listeners del NsdManager se registran y desregistran de forma explícita para evitar fugas de recursos cuando el usuario abandona la pantalla de comisionamiento.

4.5 Adaptador mDNS para localizar el simulador

MdnsSimulatorDiscoveryAdapter implementa SimulatorDiscoveryPort y resuelve el problema de configurar dinámicamente la dirección IP del simulador cuando la app se ejecuta en un **dispositivo físico conectado a la misma red** que el ordenador anfitrión.

En lugar de hardcodear una IP, el simulador anuncia su presencia con el tipo de servicio personalizado _smarthome-hub._tcp. El adaptador intenta dos vías en orden:

MdnsSimulatorDiscoveryAdapter — Localización del simulador



MdnsSimulatorDiscoveryAdapter - localización del simulador

ServiceInfoCallback es la API moderna de NsdManager (Android 13+) que evita la llamada deprecada resolveService(). La IP obtenida se persiste en SimulatorHostRepository y es usada por SimulatorDeviceList.forHost(host) para construir la lista de dispositivos con la IP correcta.

4.6 Por qué mDNS falla en el emulador

El emulador Android usa una pila de red virtualizada (QEMU) donde el tráfico multicast, necesario para mDNS, no se propaga correctamente entre la interfaz virtual del emulador y la interfaz real del ordenador anfitrión. En consecuencia:

- NsdManager no descubre los servicios anunciados por el simulador que corre en el anfitrión.
- FindOperational del SDK Matter tampoco puede localizar dispositivos ya comisionados.

Esta es la razón fundamental por la que se mantiene el adaptador estático para el entorno de desarrollo con emulador, y es también la causa de que CASE no funcione en emulador (CASE requiere FindOperational para localizar el dispositivo por su NodeId operacional antes de establecer la sesión).

4.7 Resumen de decisiones

Situación	Adaptador activo	Motivo
Emulador Android	FallbackDeviceDiscoveryAdapter → LocalhostDeviceDiscoveryAdapter	mDNS no funciona en emulador; sonda TCP confirma que el simulador está en marcha
Dispositivo físico, misma red	FallbackDeviceDiscoveryAdapter → MdnsDeviceDiscoveryAdapter	Descubrimiento real vía NsdManager antes de que expire el delay de fallback
Dispositivo físico con adb forward	FallbackDeviceDiscoveryAdapter → LocalhostDeviceDiscoveryAdapter (127.0.0.1)	Probe TCP a loopback detecta el túnel
IP del simulador desconocida	MdnsSimulatorDiscoveryAdapter	Localización automática por tipo de servicio personalizado + fallback TCP
IP del simulador ya conocida	SimulatorHostRepository (caché)	Evita redescubrimiento en cada arranque

5. Simulación de dispositivos Matter

5.1 Tecnología utilizada: matter.js

El simulador utiliza **matter.js**, la implementación JavaScript del SDK de Matter, para crear 27 nodos ServerNode independientes. Cada nodo ejecuta el protocolo Matter completo sobre UDP: anuncia su presencia por mDNS, acepta sesiones PASE y responde a comandos de

clusters estándar. Para cualquier controlador Matter, estos nodos son indistinguibles de hardware físico real.

La elección de matter.js frente a otras alternativas (simulador Python de connectedhomeip, dispositivos físicos de prueba) responde a:

- Facilidad de integración y arranque rápido (npm install + node src/main.mjs).
- Posibilidad de simular múltiples tipos de dispositivo con pocas líneas de código.
- Compatibilidad total con el protocolo Matter estándar.

5.2 Dispositivos simulados

Al arrancar, los 27 dispositivos se anuncian por mDNS como dispositivos no comisionados. Cada uno expone su **manual pairing code** y **QR code** en el log para facilitar el comisionamiento desde la app.

Bombillas inteligentes (10) - DimmableLight

#	Nombre	Puerto	Discriminador	Passcode
1	Bombilla Salon 1	5540	3840	20202021
2	Bombilla Salon 2	5541	3841	20202022
3	Bombilla Salon 3	5542	3842	20202023
4	Bombilla Cocina 1	5543	3843	20202024
5	Bombilla Cocina 2	5544	3844	20202025
6	Bombilla Dormitorio 1	5545	3845	20202026
7	Bombilla Dormitorio 2	5546	3846	20202027
8	Bombilla Baño	5547	3847	20202028
9	Bombilla Garaje	5548	3848	20202029
10	Bombilla Pasillo	5549	3849	20202030

Interruptores on/off (5) - OnOffPlugInUnit

#	Nombre	Puerto	Discriminador	Passcode
11	Interruptor Salon	5550	3850	20202031
12	Interruptor Cocina	5551	3851	20202032
13	Interruptor Dormitorio	5552	3852	20202033
14	Interruptor Baño	5553	3853	20202034
15	Interruptor Garaje	5554	3854	20202035

Cerraduras (2) - DoorLock

#	Nombre	Puerto	Discriminador	Passcode
16	Cerradura Entrada	5555	3855	20202036
17	Cerradura Garaje	5556	3856	20202037

Sensor de contacto (1) - ContactSensor

#	Nombre	Puerto	Discriminador	Passcode
18	Sensor Contacto Entrada	5557	3857	20202038

Persianas (4) - WindowCovering

#	Nombre	Puerto	Discriminador	Passcode
19	Persiana Salon	5558	3858	20202039
20	Persiana Cocina	5559	3859	20202040
21	Persiana Dormitorio	5560	3860	20202041
22	Persiana Baño	5561	3861	20202042

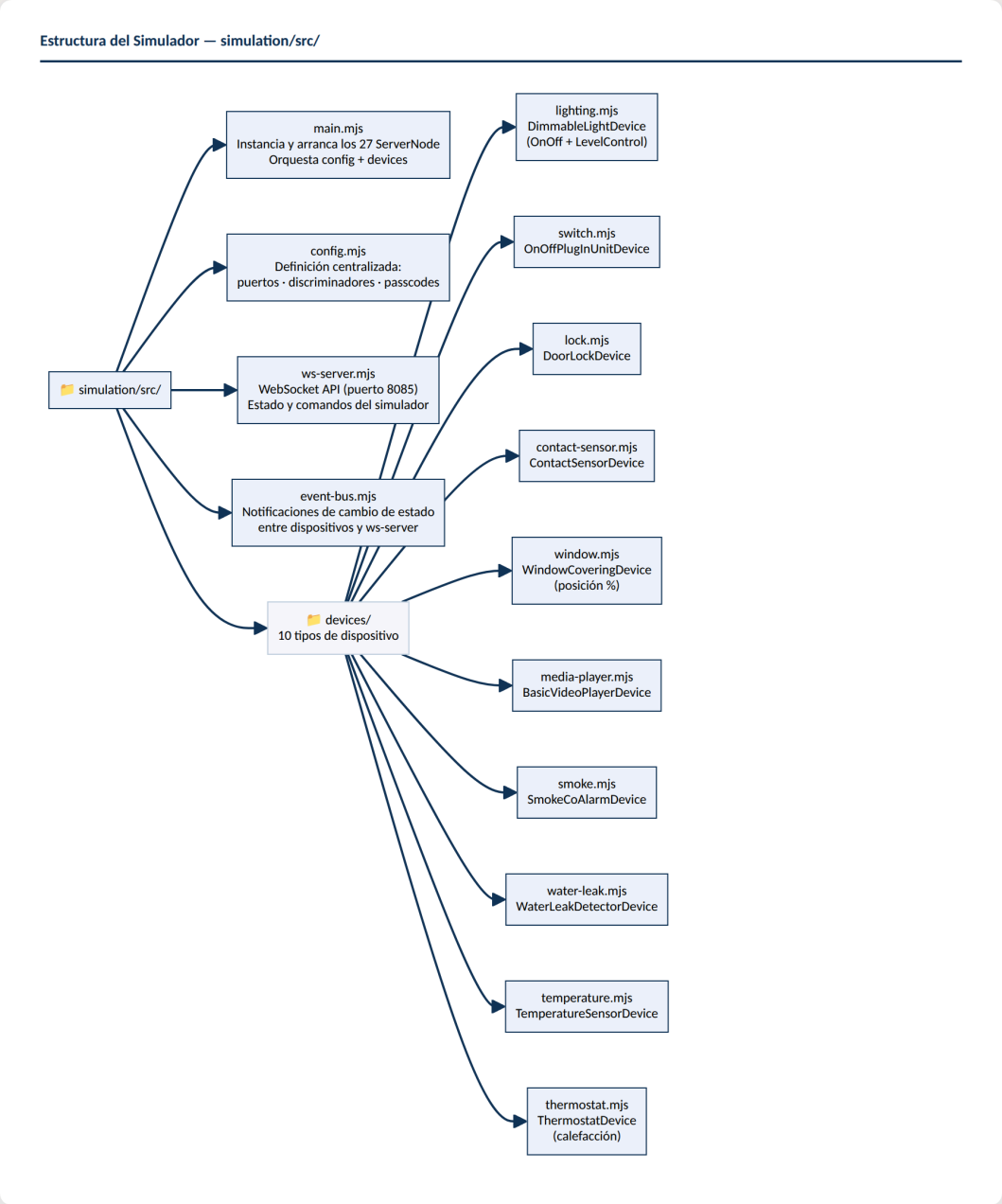
Smart TV (1) - BasicVideoPlayer

#	Nombre	Puerto	Discriminador	Passcode
23	Smart TV Salon	5562	3862	20202043

Sensores de medición (3) y termostato (1)

#	Nombre	Tipo	Puerto	Discriminador	Passcode
24	Sensor de Humo	SmokeCoAlarm	5563	3863	20202044
25	Sensor Fugas Agua	WaterLeakDetector	5564	3864	20202045
26	Sensor Temperatura	TemperatureSensor	5565	3865	20202046
27	Termostato	Thermostat (Heating)	5566	3866	20202047

5.3 Estructura del código de simulación



Estructura del simulador - simulation/src/

5.4 Arranque del simulador

El repositorio incluye launch.sh en smart-home/, que automatiza todo el proceso de arranque:

```
cd smart-home
./launch.sh
```

El script realiza las siguientes acciones en orden:

- Instala las dependencias de Node.js (npm install) si no están presentes, tanto para la simulación como para el dashboard.
- Elimina el estado de comisionamiento previo (rm -rf ~/.matter), forzando que todos los dispositivos arranquen como no comisionados. Esto garantiza un estado limpio en cada ejecución sin intervención manual.
- Lanza el simulador Matter (node src/main.mjs), que pone en escucha los 27 dispositivos en sus puertos UDP (5540–5566) y los anuncia por mDNS.
- Lanza el dashboard web en el puerto 3000.

Para detener ambos procesos basta con Ctrl+C; el script captura la señal y los para limpiamente.

6. Simulación de sensores y sistema de eventos

6.1 Por qué los sensores se simulan localmente

En el protocolo Matter, recibir lecturas continuas de un sensor requiere establecer **suscripciones a atributos** (attribute subscriptions). Este mecanismo permite que el controlador reciba notificaciones automáticas cuando un atributo del dispositivo cambia, sin necesidad de hacer polling.

Sin embargo, las suscripciones a atributos requieren **sesiones CASE** activas, ya que son operaciones de larga duración que necesitan el material criptográfico de la fabric. Dado que CASE no está soportado en el proyecto (ver sección 3.3), no es posible recibir eventos reales desde el simulador Matter.

La solución adoptada es generar los eventos de sensores **localmente dentro de la app** mediante SensorEventSimulator. Los valores no provienen del simulador externo, sino de generadores de datos simulados con patrones realistas.

6.2 SensorEventSimulator

SensorEventSimulator implementa BackgroundSimulatorPort y lanza 7 corutinas concurrentes al iniciarse, cada una responsable de un tipo de evento diferente:

Lecturas de temperatura

- **Frecuencia**: cada 3 segundos.
- **Rango**: valores aleatorios entre 18 y 28 °C.
- **Filtro**: solo se emite un evento si la variación respecto al valor anterior supera 1 °C, evitando ruido innecesario.
- **Dispositivo objetivo**: sensor de temperatura del domicilio.

Eventos de sensor de contacto (puerta)

- **Frecuencia**: toggle automático cada 10 segundos.
- **Comportamiento**: alterna entre "puerta abierta" y "puerta cerrada".

- **Efecto secundario**: además de generar el evento, modifica el estado del dispositivo ContactSensor en DeviceRepository (sensor.toggle() + deviceRepository.save()), lo que actualiza la UI en tiempo real.

Alertas de humo

- **Frecuencia**: cada 5 segundos.
- **Probabilidad de alerta**: 50 % en cada ciclo.
- **Comportamiento**: genera una alerta de humo activa o su resolución aleatoriamente.

Alertas de fugas de agua

- **Frecuencia**: cada 5 segundos.
- **Probabilidad de alerta**: 50 % en cada ciclo.
- **Comportamiento**: análogo a las alertas de humo.

Ajustes de termostato

- **Frecuencia**: cada 2 minutos.
- **Probabilidad de ajuste**: 30 % en cada ciclo.
- **Comportamiento**: genera pequeñas variaciones en la temperatura objetivo del termostato.

Monitor de duración de puerta abierta

- **Frecuencia de comprobación**: cada 2 minutos.
- **Comportamiento**: si el sensor de contacto lleva más de 2 minutos en estado "abierto", genera una alerta de puerta abierta prolongada.

Simulación de presencia (antiokupas)

- **Frecuencia**: cada 60 segundos.
- **Comportamiento**: genera eventos de presencia simulada que integran con el modo antiokupas.

6.3 Generación de IDs de eventos

Cada evento generado recibe un ID único construido con la combinación de timestamp en milisegundos más un sufijo aleatorio, garantizando unicidad incluso si dos eventos se generan en el mismo instante.

6.4 Flujo de eventos hacia la UI

Los eventos se almacenan en DeviceEventRepository (implementación en memoria) y son accesibles desde la pantalla de historial de notificaciones.

6.5 Notificaciones Android

Cuando se genera un evento de tipo alerta (humo, fuga de agua, puerta abierta prolongada), AndroidNotificationAdapter crea una notificación del sistema Android con:

- Canal dedicado por tipo de alerta (creado en el arranque de la app).
- Icono y texto descriptivo del evento.
- Prioridad alta para eventos de seguridad (humo, fugas).

Las notificaciones se generan localmente sin servidor externo, lo que las hace funcionar incluso sin conexión a internet.

7. App companion Wear OS

7.1 Propósito y funcionalidad

La app Wear OS permite controlar los dispositivos del hogar directamente desde el smartwatch mediante **acciones rápidas de un toque**, sin necesidad de desbloquear el teléfono. La pantalla principal muestra todos los dispositivos agrupados por habitación.

Acciones disponibles por tipo de dispositivo

Tipo	Acción	Indicador visual
Luz	Toggle on/off	ON / OFF
Cerradura	Lock / Unlock	Icono de candado
Interruptor	Toggle on/off	ON / OFF
Smart TV	Toggle on/off	ON / OFF
Persiana	Toggle 0% / 100%	Porcentaje
Termostato	Toggle calefacción	Temperatura
Sensores	Solo lectura	Estado actual

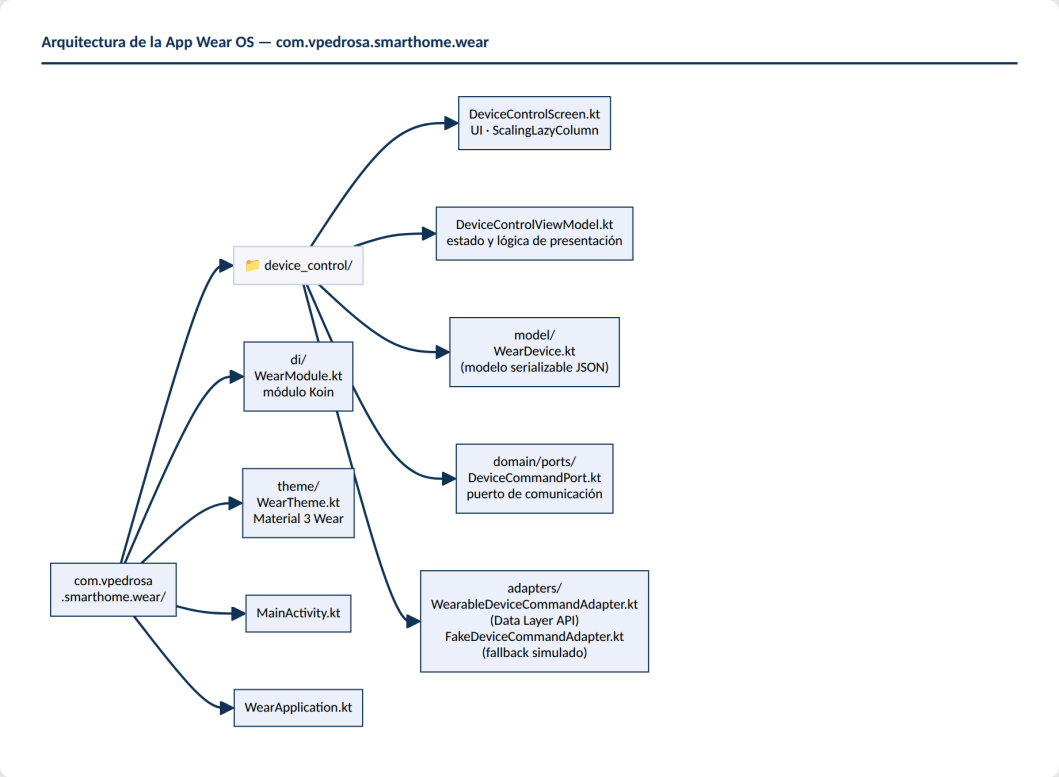
7.2 Decisión de diseño: acciones rápidas vs. control por voz

La primera versión de la app Wear OS utilizaba **control por voz** mediante SpeechRecognizer y RecognizerIntent. Esta funcionalidad se descartó por las siguientes razones:

- SpeechRecognizer no está disponible en los emuladores de Wear OS.
- RecognizerIntent resulta poco fiable en emuladores, con frecuentes timeouts.
- La gestión del audio en Wear OS introduce complejidad adicional respecto a Android estándar.
- La transmisión del texto reconocido al teléfono para su procesamiento genera latencia inaceptable.

Las **acciones rápidas** son más apropiadas para el factor de forma del smartwatch: interacción en menos de dos segundos, sin necesidad de hablar, y funcionamiento fiable en emuladores.

7.3 Arquitectura de la app Wear OS



Arquitectura de la app Wear OS

La app del reloj también sigue la arquitectura hexagonal, aunque más simplificada dado que no contiene lógica de negocio propia: delega toda la lógica al teléfono a través del protocolo de mensajes.

7.4 Protocolo de comunicación Watch–Phone

La comunicación se realiza mediante la **Wearable Data Layer API** (MessageClient) de Google. Es un protocolo de mensajes asíncronos, no un canal persistente.

Mensajes del protocolo

Dirección	Path	Payload
Reloj → Teléfono	/device_list_request	(vacío)
Teléfono → Reloj	/device_list_response	JSON: {"devices": [...]}
Reloj → Teléfono	/device_action	JSON: {"deviceId": "x", "action": "TOGGLE"}
Teléfono → Reloj	/device_action_result	OK:{device JSON} o ERROR:mensaje

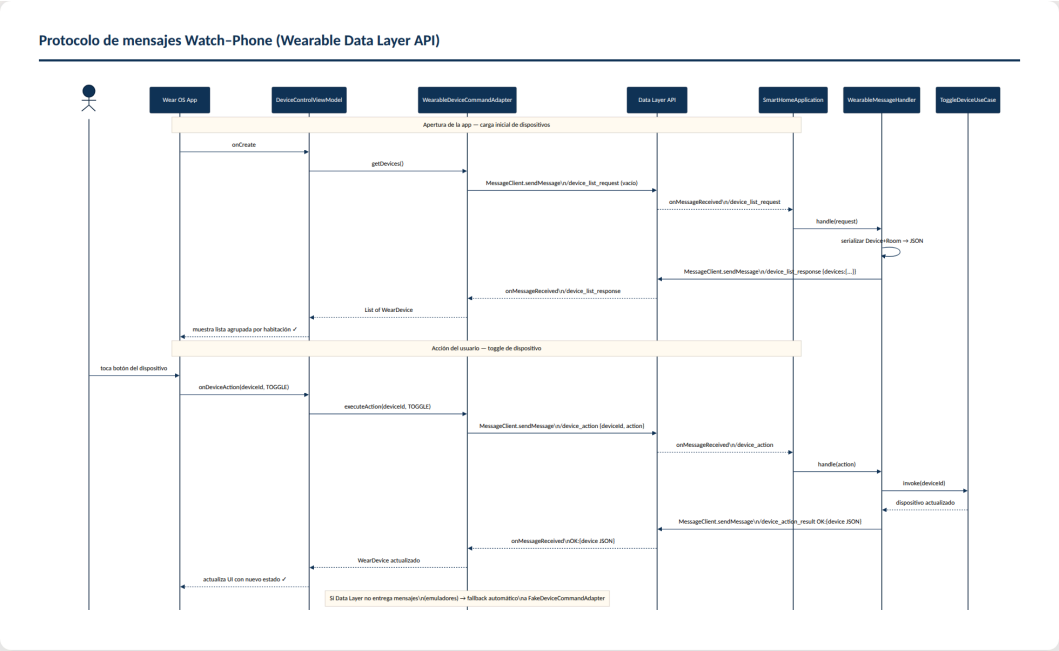
Flujo completo de una interacción

7.5 Recepción en el teléfono

El teléfono tiene dos mecanismos para recibir mensajes del reloj:

- **WearableMessageListenerService**: servicio en segundo plano declarado en el manifiesto, activo incluso cuando la app no está en primer plano.
- **WearableMessageHandler** registrado en SmartHomeApplication: listener de primer plano como complemento para entornos de emulación.

Ambos mecanismos delegan al mismo WearableMessageHandler, que usa los casos de uso existentes (ToggleDeviceUseCase, DeviceRepository, RoomRepository). No existe duplicación de lógica.



Protocolo de mensajes Watch–Phone (Wearable Data Layer API)

7.6 Fallback con datos simulados

Si la comunicación con el teléfono falla (timeout, emuladores sin Data Layer activo), la app del reloj conmuta automáticamente a FakeDeviceCommandAdapter, que proporciona datos simulados. Esto permite visualizar y navegar la UI del reloj en un emulador sin un teléfono emparejado.

7.7 Limitaciones conocidas

- La **Data Layer API no entrega mensajes entre emuladores** de forma fiable. NodeClient.connectedNodes puede mostrar conexión establecida, pero MessageClient no enruta los mensajes correctamente entre instancias de emulador.
- En **dispositivos físicos** (teléfono + reloj real) la comunicación funciona correctamente.
- La app Wear OS requiere emulador con **Wear OS API 35** o superior.

8. Control por voz

8.1 Diseño del sistema de reconocimiento

El control por voz utiliza el **SpeechRecognizer nativo de Android** para la captura y transcripción del audio. La elección de la API nativa frente a soluciones de terceros responde a:

- No requiere conexión a internet (modo offline disponible en dispositivos con paquete de idioma descargado).
- Sin dependencias externas ni claves de API.
- Integración directa con el sistema operativo Android.

El flujo está dividido en dos fases claramente separadas: **reconocimiento de voz** (infraestructura Android) y **parsing + ejecución** (lógica de dominio pura, sin dependencias de Android).

8.2 Reconocimiento de voz: AndroidSpeechRecognizerAdapter

AndroidSpeechRecognizerAdapter implementa SpeechRecognizerPort y expone tres Flows:

Flow	Tipo	Descripción
isListening	Flow<Boolean>	Indica si el micrófono está activo
recognizedText	Flow<String>	Texto reconocido cuando finaliza la escucha
errors	Flow<String>	Mensajes de error legibles

Detalles de configuración:

- Todas las operaciones se ejecutan en el hilo principal (Handler(Looper.getMainLooper())), requisito de la API de Android.
- El modelo de reconocimiento es LANGUAGE_MODEL_FREE_FORM para lenguaje natural.
- El idioma configurado es es-ES (español de España).
- Los resultados parciales están desactivados para obtener el texto completo de una vez.
- El SpeechRecognizer se recrea en cada sesión para evitar estado obsoleto.

8.3 Parsing de comandos: ParseVoiceCommandUseCase

ParseVoiceCommandUseCase convierte texto libre en un ParsedVoiceCommand mediante **coincidencia de patrones con expresiones regulares**. No utiliza ningún modelo de lenguaje externo ni procesamiento de lenguaje natural con ML.

El texto se normaliza antes del parsing:

- Conversión a minúsculas.
- Eliminación de tildes y caracteres diacríticos (ej. "enciéndelas" → "enciendelas"), lo que hace el matching robusto ante variaciones de escritura.

Comandos soportados

Español:

Tipo	Palabras clave de activación	Palabras clave de desactivación
Luces	enciende, activa, prende, ilumina	apaga, desactiva, apaga
Persianas	abre (persianas)	cierra (persianas)
Cerraduras	abre (puerta/cerradura)	cierra (puerta/cerradura)
Termostato	pon/sube/ajusta la temperatura a X grados	-
TV	enciende la televisión/tele	apaga la televisión/tele
Interruptores	enciende los interruptores	apaga los interruptores

Inglés:

Tipo	Activación	Desactivación
Lights	turn on, switch on, activate	turn off, switch off, deactivate
Blinds	open (blinds)	close (blinds)
Locks	unlock (door)	lock (door)
Thermostat	set/change/adjust temperature to X degrees	-

Extracción de habitación

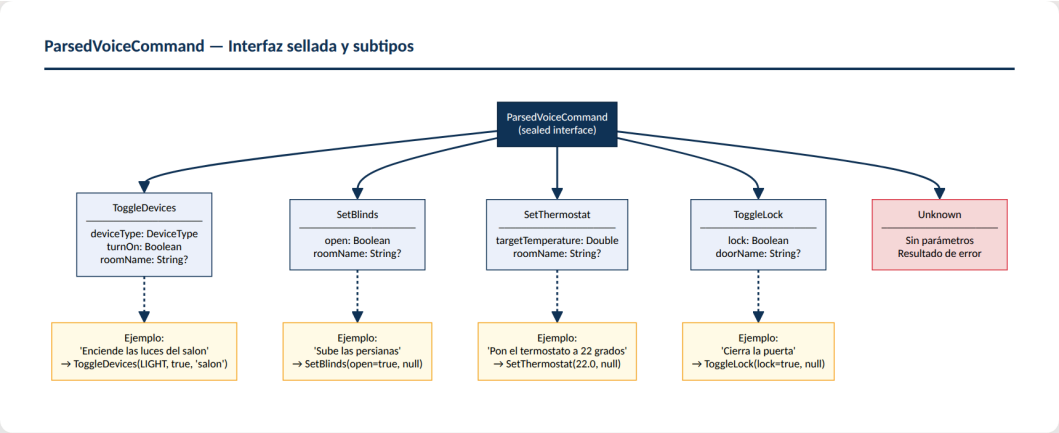
El parser extrae el nombre de la habitación del comando usando patrones como:

- Español: "del/de la/en el/en la [habitación]"
- Inglés: "in the/in [room]"

Si no se especifica habitación, el comando se aplica a todos los dispositivos del tipo indicado.

Resultado del parsing

ParsedVoiceCommand es una interfaz sellada con los siguientes subtipos:



ParsedVoiceCommand - interfaz sellada y subtipos

8.4 Ejecución: ExecuteVoiceCommandUseCase

ExecuteVoiceCommandUseCase recibe un ParsedVoiceCommand y lo ejecuta invocando los casos de uso de control correspondientes:

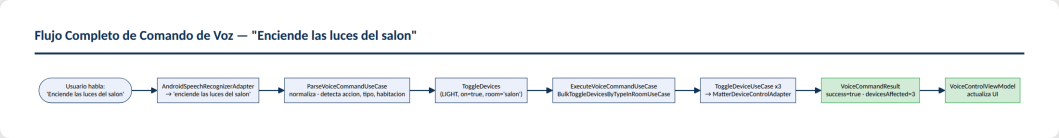
Comando	Caso de uso invocado
ToggleDevices sin habitación	BulkToggleDevicesByTypeUseCase
ToggleDevices con habitación	BulkToggleDevicesByTypeInRoomUseCase
SetBlinds	deviceControlPort.setWindowCoveringPosition(0% o 100%)
SetThermostat	deviceControlPort.setThermostatSetpoint(temperatura)
ToggleLock	deviceControlPort.lockDoor(estado)
Unknown	Devuelve resultado de error sin ejecutar nada

El uso devuelve un VoiceCommandResult con:

- success: si se ejecutó al menos una acción.
- message: descripción legible del resultado.
- devicesAffected: número de dispositivos modificados.

Si no se encuentra ningún dispositivo del tipo o habitación indicados, el resultado indica fallo sin lanzar excepción.

8.5 Ejemplo de flujo completo



Flujo completo de comando de voz

9. Funcionalidades principales

9.1 Control de dispositivos

La app permite controlar cada tipo de dispositivo con sus acciones específicas:

Tipo	Acciones disponibles
Bombilla (DimmableLight)	Toggle on/off, ajuste de brillo (0-100%), selección de color
Interruptor (OnOffPlugInUnit)	Toggle on/off
Cerradura (DoorLock)	Lock / Unlock
Persiana (WindowCovering)	Posición continua (0-100%), apertura/cierre rápido
Smart TV (BasicVideoPlayer)	Toggle on/off
Termostato (Thermostat)	Temperatura objetivo, toggle modo calefacción
Sensores	Solo lectura (temperatura, estado de contacto, alertas)

Cada tipo de dispositivo tiene su propia pantalla de detalle con controles adaptados a sus capacidades.

Escalas de traducción Matter

Los valores de la UI se traducen a las escalas internas del protocolo Matter antes de enviarse:

Parámetro	Escala UI	Escala Matter
Brillo de bombilla	0–100%	0–254 (LevelControl cluster)
Posición de persiana	0–100%	0–10000 (WindowCovering cluster)
Temperatura del termostato	°C (ej. 22.0)	1/100 °C (ej. 2200)

9.2 Gestión de habitaciones

Los dispositivos se organizan en habitaciones. La app incluye las siguientes habitaciones predefinidas: salón, cocina, dormitorio, baño, garaje y pasillo. La UI permite crear, editar y eliminar habitaciones, así como asignar dispositivos a ellas.

La agrupación por habitación es puramente lógica en la app: el protocolo Matter no tiene un concepto nativo de habitación; es la capa de aplicación quien gestiona esta organización.

9.3 Modo antiokupas

El **modo antiokupas** simula presencia en el hogar cuando los ocupantes no están. Activa y desactiva dispositivos (luces, TV) de forma programada con patrones aleatorios para imitar comportamiento humano real y disuadir posibles intrusiones.

El AntisquatterSimulator lanza corutinas que, cada 60 segundos, deciden aleatoriamente si encender o apagar dispositivos seleccionados, generando el patrón de uso que resultaría natural en una vivienda ocupada.

9.4 Notificaciones y alertas

La app genera notificaciones locales del sistema Android para los siguientes eventos:

- **Alerta de humo**: cuando el sensor de humo detecta activación.
- **Alerta de fuga de agua**: cuando el sensor de fugas detecta agua.
- **Puerta abierta prolongada**: cuando el sensor de contacto lleva más de 2 minutos en estado abierto.

Cada tipo de alerta usa su propio canal de notificación Android, lo que permite al usuario configurar la importancia y el sonido de cada categoría de forma independiente desde los ajustes del sistema.

La pantalla de eventos de la app muestra el historial completo de notificaciones con timestamp y estado del dispositivo en el momento del evento.

9.5 Internacionalización

La app está disponible en **español e inglés**. La localización cubre:

- Nombres de pantallas y etiquetas de UI.
- Mensajes de error y confirmación.
- 23 strings de accesibilidad (contentDescription) para lectores de pantalla.
- Los comandos de voz soportan ambos idiomas de forma nativa (ver sección 8).

9.6 Dashboard principal

La pantalla de inicio (Dashboard) muestra:

- Resumen del estado del hogar: número de dispositivos encendidos, alertas activas.
- Acceso rápido a habitaciones.
- Últimas notificaciones.
- Acceso directo al modo antiokupas y al control por voz.

9.7 Tests unitarios

La suite de tests cubre más de 93 casos de prueba, ejecutables en la JVM sin infraestructura Android:

Área	Tests
Casos de uso de dispositivos	Toggle, brillo, posición, thermostat
Control por voz	Parsing de comandos ES e EN, ejecución
Eventos y sensores	Generación y almacenamiento de eventos
Antiokupas	Lógica de simulación de presencia
Repositorios compartidos	CRUD en memoria

Los tests usan implementaciones fake de todos los puertos (repositorios en memoria, adaptador Matter fake), lo que los hace completamente deterministas y rápidos.

10. Resumen de decisiones técnicas

Esta sección recopila las decisiones de diseño más relevantes tomadas durante el desarrollo, junto con su justificación.

10.1 Kotlin Multiplatform sobre desarrollo nativo separado

Decisión: usar KMP + Compose Multiplatform para compartir lógica de negocio y UI entre Android y Wear OS, con soporte compilable para iOS (sin integración Matter en iOS, ya que el SDK de Matter solo existe en androidMain).

Justificación: una sola base de código para la lógica de dominio y de aplicación reduce la duplicación y garantiza consistencia entre plataformas. Compose Multiplatform permite además compartir la mayor parte de la UI. El coste es una mayor complejidad de

configuración del build (Gradle, source sets), pero el beneficio en mantenibilidad a largo plazo supera ese coste.

10.2 Arquitectura hexagonal con vertical slicing

Decisión: cada funcionalidad es un slice vertical independiente con sus propias capas de dominio, aplicación e infraestructura.

Justificación: la arquitectura hexagonal garantiza que la lógica de negocio no depende de frameworks ni de infraestructura. El vertical slicing evita el acoplamiento horizontal entre funcionalidades. El resultado son más de 93 tests unitarios que se ejecutan en la JVM en segundos, sin necesidad de emulador Android.

10.3 PASE en lugar de CASE para la comunicación Matter

Decisión: usar únicamente sesiones PASE (no CASE) para comunicarse con el simulador.

Justificación: CASE requiere FindOperational (mDNS) y certificados NOC, ninguno de los cuales funciona en el emulador Android con el SDK de demostración. PASE es suficiente para demostrar la integración con el protocolo Matter en el contexto del proyecto.

Consecuencia: las sesiones expiran y deben re-establecerse. El mecanismo de retry con nodeId temporal resuelve este problema de forma transparente para el usuario.

10.4 NodeId temporal para forzar nuevas sesiones PASE

Decisión: en el retry tras expiración de sesión, usar un nodeId incremental (≥100000) en lugar del nodeId original del dispositivo.

Justificación: ChipDeviceController reutiliza la sesión existente si se llama con el mismo nodeId, aunque el simulador ya la haya descartado. El nodeId temporal fuerza la creación de una sesión completamente nueva. El simulador acepta cualquier nodeId; lo que valida es el passcode.

10.5 Simulación local de eventos de sensores

Decisión: generar los eventos de temperatura, humo, fugas y contacto localmente en la app, sin leerlos del simulador Matter.

Justificación: leer valores reales de sensores requiere suscripciones a atributos Matter, que a su vez necesitan sesiones CASE. Al no soportarse CASE, la alternativa es la simulación local con SensorEventSimulator. Los valores generados son suficientemente realistas para demostrar la funcionalidad de alertas y notificaciones.

10.6 Tres adaptadores de descubrimiento intercambiables

Decisión: implementar tres adaptadores que implementan el mismo puerto DeviceDiscoveryPort: estático, mDNS real y mDNS para el simulador.

Justificación: el emulador Android no propaga mDNS hacia el anfitrión, por lo que se necesita un adaptador estático para el entorno de desarrollo. En dispositivos físicos, el adaptador mDNS real (MdnsDeviceDiscoveryAdapter) descubre los dispositivos correctamente. MdnsSimulatorDiscoveryAdapter resuelve el problema de la IP dinámica del simulador cuando se usa con dispositivo físico.

10.7 Acciones rápidas en Wear OS en lugar de control por voz

Decisión: la app Wear OS usa botones de acción rápida en lugar del control por voz originalmente previsto.

Justificación: SpeechRecognizer no está disponible en los emuladores de Wear OS. RecognizerIntent es poco fiable y genera timeouts frecuentes. Las acciones rápidas son más apropiadas para el factor de forma del reloj: interacción rápida, sin necesidad de hablar, funcionamiento garantizado en emuladores.

10.8 Parsing de voz mediante regex sin ML

Decisión: usar expresiones regulares con normalización de texto para parsear comandos de voz, sin modelos de lenguaje ni APIs de NLU externas.

Justificación: el dominio de comandos es acotado y estructurado. Las regex con normalización de diacríticos cubren las variaciones naturales del lenguaje de forma suficiente. No introduce dependencias externas, funciona offline y es completamente testeable con tests unitarios deterministas.

10.9 Timeout explícito de 3 segundos para comandos Matter

Decisión: configurar un timeout de 3 segundos para los comandos Matter en lugar de depender del comportamiento por defecto del SDK.

Justificación: sin timeout explícito, el SDK realiza hasta 4 reintentos internos con backoff exponencial antes de fallar, lo que puede resultar en esperas de ~15 segundos. El timeout de 3 segundos garantiza que el mecanismo de retry con sesión nueva se activa en tiempo razonable, mejorando la experiencia de usuario.

10.10 No actualizar estado en memoria si falla Matter

Decisión: si el comando Matter falla (incluso después del retry), el estado del dispositivo no se actualiza en DeviceRepository.

Justificación: mantener la consistencia entre el estado en memoria de la app y el estado real del simulador es prioritario sobre la velocidad de respuesta de la UI. Un optimistic update fallido que no se revierte generaría una UI que muestra un estado diferente al real del dispositivo.

11. Integración continua (CI/CD)

11.1 Pipeline de GitHub Actions

El proyecto dispone de un pipeline de integración continua en .github/workflows/smart-home-ci.yml que se activa automáticamente ante cambios en el código de la aplicación.

Eventos de activación

El pipeline se ejecuta en tres situaciones:

- **Push a master** que afecte a smart-home/app/** o al propio fichero de workflow.
- **Pull request hacia master** con los mismos filtros de ruta.
- **Ejecución manual** (workflow_dispatch) desde la interfaz de GitHub Actions.

El filtro de rutas evita que cambios en el simulador, el prototipo o la documentación disparen compilaciones innecesarias.

11.2 Jobs

El pipeline define dos jobs secuenciales:

Job test - Tests unitarios

Se ejecuta sobre ubuntu-latest con JDK 17 (Temurin). Lanza los tests de ambos módulos de la app:

```
./gradlew composeApp:testDebugUnitTest wearApp:testDebugUnitTest --no-daemon
```

Gracias a la arquitectura hexagonal, todos los tests corren en la JVM sin necesidad de emulador Android ni dispositivo físico. El job siempre sube los informes HTML de test como artefacto (test-reports), incluso si algún test falla, facilitando la revisión de errores.

Job build - Compilación de APKs

Solo se ejecuta si test ha pasado (needs: test). Compila los dos APKs en modo debug:

Tarea Gradle	Módulo	Artefacto generado
androidApp:assembleDebug	App teléfono	smart-home-phone-apk
wearApp:assembleDebug	App Wear OS	smart-home-wear-apk

Ambos APKs se suben como artefactos descargables desde la ejecución de GitHub Actions.

11.3 Decisiones de diseño del pipeline

Sin firma de release: los APKs se generan en modo debug. Para un proyecto académico no es necesario gestionar keystores ni secrets de firma.

Sin emulador: los tests no requieren Android gracias a la separación dominio/infraestructura. Esto hace el job test muy rápido (típicamente menos de 2 minutos) y sin las incidencias de estabilidad habituales en emuladores de CI.

Gradle Actions Cache: gradle/actions/setup-gradle@v4 activa automáticamente la caché de dependencias Gradle entre ejecuciones, reduciendo el tiempo de descarga en runs sucesivos.

Artefactos de test siempre disponibles: la directiva if: always() en el upload de reportes garantiza que, incluso ante un fallo de compilación o test, los informes están disponibles para diagnóstico.